

SYSTEM IMPLEMENTATION, TESTING AND RESULTS

Project: Online Food Ordering System for the University of Africa, Toru-Orua, Sagbama, Bayelsa State (implemented as "CampusBites")

[Insert your name, matriculation number, department and supervisor here per your school's cover-page format — not fabricated by this report.]

Test run date: 8 August 2026 · All results in this chapter were captured from a live, running instance of the application (see §4.7 methodology note).

4.1 Introduction

This chapter describes how the design presented in Chapter Three was implemented, the tools and environment used to build and run the system, and the testing carried out to verify that the implemented system satisfies the functional and non-functional requirements defined in Chapter Three. Every test result reported in this chapter was obtained by executing an automated test driver (`testing/run-tests.js`) against the actual running application over HTTP, and by direct inspection of the application source code and its persisted data store. No test result in this chapter has been invented or estimated; where a requirement from Chapter Three could not be exercised because the corresponding feature does not exist in the current implementation, this is stated explicitly rather than assumed or simulated.

Section 4.2–4.6 describe the implemented system as built. Section 4.7 presents the testing methodology and results (unit, integration, system, negative, performance and security testing, plus a synthetic usability evaluation). Section 4.8 discusses the results, reconciles them with the Chapter Three design, and maps each functional and non-functional requirement to the specific tests that exercised it.

4.2 Development Environment

Component	Detail
Operating system (development/test host)	Windows 10 Pro
Runtime	Node.js v24.18.0
Package manager	npm 11.16.0
Web server (dev)	Express application served locally at <code>http://localhost:3000</code>

Version control	Not initialised as a Git repository at the time of this test run
Test execution method	Node.js script issuing real HTTP requests (<code>fetch</code>) against the running server, with manual session-cookie handling to simulate concurrent student, vendor and admin sessions

4.3 Programming Languages, Frameworks and Tools Used

Layer	Technology actually used in the codebase
Server-side language	JavaScript (Node.js, CommonJS modules)
Web framework	Express.js 4.19
Server-rendered templating	EJS 3.1 (<code>views/*.ejs</code>)
Styling	Tailwind CSS 3.4, compiled at build time (<code>npm run build:css → public/css/app.css</code>) — not loaded from a CDN
Client-side interactivity	Vanilla JavaScript <code>fetch()</code> calls embedded directly in EJS views; no client-side framework (no React/Vue/Angular)
Authentication	<code>express-session</code> (cookie-based server sessions) + <code>bcryptjs</code> for password hashing
Data persistence	A custom flat-file JSON store (<code>data/store.js</code>) reading/writing <code>data/db.json</code> — see §4.4 for the important discrepancy this represents against the Chapter Three design
Icons	Lucide icon set, referenced via CDN <code><script></code> tag in <code>views/partials/header.ejs</code>
Dev/test dependency present but unused for this run	<code>playwright-core</code> is listed in <code>devDependencies</code> but no browser binary was installed in this environment, so it was not used for test automation; the HTTP-level driver described in §4.7 was used instead

4.4 Database Implementation

Important discrepancy from the Chapter Three design, confirmed by direct source inspection: Chapter Three specifies SQLite as the database technology, and `SPEC.md` in the project root also documents "SQLite via `better-sqlite3`". Neither is actually used. `better-sqlite3` is not present in `package.json`, and there is no `.sqlite/.db` file anywhere in the project. The application instead persists all data in a single JSON file, `data/db.json`, through a small hand-written module, `data/store.js`, which exposes `load()`, `save()` and a synchronous `tx()` helper that reads the whole file, mutates it in memory, and writes it back. This is functionally a document-store implementation of the same five-entity relational design from Chapter Three (users, vendors, menu items, orders, order items), not a relational SQL database. This chapter reports this exactly as found; Chapter Three / the project proposal should be

corrected to describe the database layer accurately, or the implementation should be migrated to SQLite before final submission if the examiners will check the code against the written design.

4.4.1 Entities actually implemented (as fields in `data/db.json`)

Chapter 3 entity	Actual JS object fields (in <code>data/store.js</code> tables)	Match to Chapter 3 design?
Users user_id, full_name, email, password_hash, role, date_registered	id, name, email, passwordHash, role, status, createdAt	Core fields match (renamed to camelCase). An additional status field exists (active/pending/rejected) that Chapter 3's user entity does not list — it is what actually drives vendor approval.
Vendors vendor_id, user_id, business_name, vendor_type, approval_status	id, userId, businessName, description	No vendor_type field exists (no "outlet" vs "student_business" distinction is stored — it can only be inferred informally from the free-text description). No approval_status field on the vendor record — approval status actually lives on the linked <i>user's</i> status field, with values active / pending / rejected (not Chapter 3's pending/approved/suspended).
Orders order_id, student_id, vendor_id, order_date, status, total_amount	id, studentId, vendorId, status, createdAt	No total_amount is stored on the order. Total is computed on demand by summing order_items each time it is displayed (routes/student.js). Status values implemented: pending → preparing → ready → completed. No declined status exists — there is no order-rejection/cancellation feature (confirmed against SPEC.md: "No order cancellation").
Menu Items item_id, vendor_id, item_name, price, availability	id, vendorId, name, price	No availability field exists. Every seeded menu item is implicitly always orderable; there is no way, in the current implementation, to mark an item unavailable/out of stock, and no category field either.

Because the running application has no `vendor_type`, `approval_status` (as named in Ch. 3), `total_amount`, or `availability` columns, the test dataset in §4.7 uses Chapter-3-style column headers in its CSV exports for readability (e.g. `total_amount` is a computed column, not a stored one) but does not claim these are literal database columns. This is documented per-file in `testing/test-data/`.

4.5 System Implementation

The system follows a three-tier structure consistent with Chapter Three, implemented concretely as:

- **Presentation layer:** EJS templates in `views/`, rendered server-side per request, styled with compiled Tailwind CSS, with small `fetch()`-based polling scripts for the two live-updating views (order tracking and vendor dashboard).
- **Application logic layer:** Express route handlers, split by role into `routes/auth.js` (registration/login/logout, shared), `routes/student.js` (vendor browsing, cart submission, order tracking), `routes/vendor.js` (incoming-order dashboard, status advancement), and `routes/admin.js` (vendor approval/rejection, platform stats). Role-based access is enforced by a single reusable `requireRole(...roles)` middleware in `middleware/auth.js`.
- **Data layer:** `data/store.js`, as described in §4.4.

4.5.1 Order status lifecycle as implemented

Implemented exactly as a fixed forward-only state machine in `routes/vendor.js`:

```
pending → preparing → ready → completed
```

Each call to `POST /vendor/orders/:id/advance` moves the order exactly one step along this chain (looked up via a `NEXT_STATUS` map); there is no way to skip a step, move backwards, or cancel/decline an order once placed.

4.5.2 Notification mechanism (FR4)

There is no push mechanism (no WebSockets, no email, no SMS) — this matches `SPEC.md`'s explicit scope cut. "Notification" is implemented as short-interval client-side polling: `views/vendor-dashboard.ejs` and `views/orders.ejs` both call their respective JSON endpoint (`/api/vendor/orders`, `/api/orders/mine`) immediately on page load and then every 4000ms via `setInterval`, re-rendering the list from scratch each time.

4.5.3 Payment

No payment gateway integration exists. The order flow ends at "Pay on pickup" as a business convention only — no payment record, status, or amount-due field is created or tracked anywhere in the code.

4.6 User Interface Design

Nine server-rendered pages/routes were confirmed present, matching the Chapter Three page list:

Route	View file	Purpose
/	landing.ejs	Landing page
/register	register.ejs	Role-aware registration form, with client-side email-format and password-strength hints (see §4.7.4 for a finding about the server not re-checking these)
/login	login.ejs	Login form
/vendors	vendors.ejs	Student: card grid of active vendors
/vendors/:id	vendor-menu.ejs	Student: single vendor's menu, add to cart
/orders	orders.ejs	Student: order history, polling every 4s
/vendor/dashboard	vendor-dashboard.ejs	Vendor: incoming orders by status, polling every 4s, one-tap advance
/admin/dashboard	admin-dashboard.ejs	Admin: pending approvals, active vendor table, platform counters

Screenshot evidence to capture manually for this section (see §4.8.6 for the full list): landing page, registration form (student and vendor toggle states), vendor grid, a vendor menu page, the cart/order confirmation, the student order-tracking page mid-poll, the vendor dashboard with active orders, and the admin dashboard with a pending approval visible.

4.7 System Testing

Methodology

All tests below were executed by `testing/run-tests.js`, a Node.js script that issues real HTTP requests against the running server (freshly reset with `npm run seed` beforehand) and records the actual HTTP status codes, redirect targets, and response bodies returned. Session cookies were captured and replayed per simulated user so that 11 concurrent student sessions, 11 vendor sessions and 1 admin session could be driven independently, exactly as separate browser tabs would behave. A small number of bookkeeping reads of `data/db.json` were used only to look up internal IDs needed to construct follow-up requests (e.g. which user ID to approve) and to verify persisted state after a request — never to fabricate a result. Full source: `testing/run-tests.js`. Full raw output: `testing/results/*.csv`.

```
Pass Rate (%) = (Number of Passed Tests / Total Number of Scored Tests) × 100
```

"Scored" excludes tests marked *N/A*, which are cases where the requirement being probed corresponds to a feature that does not exist in the codebase (see §4.4 and §4.5) and is therefore not executable — these are reported as findings, not silently passed or failed.

4.7.1 Unit Testing

Table 4.1 — Unit Testing Results (21 tests; 20 passed, 1 failed, 95.2% pass rate)

ID	Module/Function	Test Input	Expected Result	Actual Result	Status
T001	Authentication — login()	Valid admin credentials	Session created, redirect to /admin/dashboard	HTTP 200, Admin dashboard rendered	PASS
T002–T011	Registration — register() [student] ×10	10 distinct valid student name/email/password combinations	Account created, auto-logged-in, redirected to /vendors	All 10: initial 302 → final 200	PASS (10/10)
T016	Registration — register() [vendor]	businessName="Northgate Grills", role=vendor	Account created with status=pending, not auto-logged-in, approval notice shown	Created, status=pending, notice shown	PASS

T017	Registration — register() [vendor]	businessName="Ada's Small Chops", role=vendor	Same as above	Created, status=pending, notice shown	PASS
T025	Menu Browsing — GET /vendors	Authenticated student request	Only active vendors listed	10 active vendors listed; rejected vendor correctly absent; approved vendor correctly present (rendered with HTML-entity- escaped apostrophe by EJS)	PASS
T026	Menu Browsing — GET /vendors/:id	Valid vendor ID	Menu items with names/prices rendered	HTTP 200, business name present in page	PASS
T028	Order Placement — POST /orders (bulk)	40 realistic multi-item carts across 10 vendors / 11 students	Each order + its order_items persisted, status=pending on creation	40/40 created successfully	PASS
T029	Order total calculation	Spot-check of 5 placed orders	sum(price_at_order × quantity) matches snapshot price	Recomputation from stored order_items consistent for all checked orders	PASS
T034	Order Tracking — GET /api/orders/mine	Authenticated student request	JSON list of only that student's own orders	HTTP 200, correct order set returned	PASS
T037	Order Status Update — POST /vendor/orders/:id/advance	40 orders advanced 0–3 steps each	Status advances exactly one step per call; final distribution spans all 4 statuses	Final distribution: 10 pending / 10 preparing / 10 ready / 10 completed	PASS
T041	Admin reporting — stats on GET /admin/dashboard	Admin loads dashboard after test data generated	Counters reflect live db.json state	40 total orders, 11 students — dashboard numerically consistent	PASS
T042	Vendor Menu Management (FR7)	Search of routes/vendor.js + vendor-dashboard.ejs for CRUD routes	Chapter 3 FR7 expects add/edit/remove menu items	NOT IMPLEMENTED — no route or UI exists to create, edit or delete a menu item; menus are fixed at seed time only	FAIL

Full 21-row table with every individual student registration row:
 testing/results/unit_test_results.csv.

4.7.2 Integration Testing

Table 4.2 — Integration Testing Results (3 tests; 3 passed, 100%)

ID	Flow	Input	Expected Result	Actual Result	Status
T021	Admin → Vendor Approval → Vendor Access	Admin approves pre-seeded pending vendor "Bola's Snack Corner"	user.status: pending → active	status now "active"	PASS
T023	Admin → Vendor Rejection → Access Revoked	Admin rejects "Ada's Small Chops"	user.status → rejected; vendor row + menu items deleted	status="rejected"; vendor row removed	PASS
T036	Student → Order → Vendor (poll-based notification)	Order placed by a student for a specific vendor	Order appears in that vendor's GET /api/vendor/orders feed without manual refresh	Order present in vendor feed	PASS

4.7.3 System Testing

Table 4.3 — End-to-End System Testing Results (5 scenarios; 2 passed, 1 failed, 2 not testable)

ID	Scenario	Steps	Expected	Actual	Status
T022	TEST 6: Admin approves vendor → vendor gains access	Register vendor → admin approves → vendor logs in → dashboard loads	Login succeeds post-approval, dashboard renders	Confirmed — vendor dashboard rendered with correct business name	PASS
T038	TEST 2: Vendor updates status → student sees it	Order placed → vendor advances it → student re-polls /api/orders/mine	Student-facing status reflects vendor's update	Student saw status="preparing" after vendor action, no manual student action needed	PASS
T052	TEST 3/4/5: Vendor adds/edits/removes a menu item	Searched for a menu-management route/form	N/A per Ch.3 FR7	NOT TESTABLE — feature does not exist (see T042)	<i>N/A</i>
T053	TEST 7: Admin generates a sales report	Loaded /admin/dashboard as admin	Ch.3 FR8 mentions "generate basic sales reports"	Dashboard shows live counters + a per-vendor order-count column only; no revenue total, no date filter, no exportable report artifact	<i>N/A / PARTIAL</i>
T054	TEST 8: Invalid registration data submitted	POST /register with email="not-an-email"	Server-side rejection expected	Account was created — email format is validated only in client-side JS (register.ejs), not	FAIL

				re-checked server-side (routes/auth.js)	
--	--	--	--	--	--

4.7.4 Negative / Error Testing

Table 4.4 — Negative/Error Testing Results (12 tests; 12 passed, 100%)

ID	Scenario	Input	Expected	Actual	Status
T012	Duplicate email registration	Re-register an already-used email	Rejected: "account already exists"	Rejected as expected	PASS
T013	Missing required fields	name/email/password all blank	Rejected: "All fields are required."	Rejected as expected	PASS
T014	Invalid role value	role="superadmin"	Rejected: "Invalid role."	Rejected as expected	PASS
T015	Vendor registration, no business name	role=vendor, businessName missing	Rejected: "Business name is required for vendors."	Rejected as expected	PASS
T018	Login while vendor account still pending	Valid creds, status=pending	Rejected: "still pending admin approval"	Rejected as expected	PASS
T019	Login with wrong password	Correct email, wrong password	Rejected: "Invalid email or password."	Rejected as expected	PASS
T020	Login with non-existent email	Unregistered email	Rejected: "Invalid email or password."	Rejected as expected	PASS
T027	Menu for non-existent vendor ID	GET /vendors/999999	404 "Vendor not found"	404 as expected	PASS
T030	Order with empty cart	items=[]	400 "Cart is empty"	400 as expected	PASS
T031	Order with malformed cart JSON	items="not-json"	400 "Invalid cart data"	400 as expected	PASS
T032	Order for non-existent vendor	vendorId=999999	400 "Vendor not found"	400 as expected	PASS
T040	Advance a non-existent order ID	POST /vendor/orders/999999/advance	No crash; silent no-op	HTTP 200 after redirect, no crash, no explicit 404 handling	PASS

4.7.5 Performance Testing

Each operation below was invoked repeatedly against the live local server and timed client-side with `performance.now()`, wall-clock, including full request/response round-trip. These are localhost, single-machine, unloaded timings — they characterise application-layer overhead, not production network latency or behaviour under concurrent load.

Average Response Time = $(\sum T_i) / n$, for $i = 1 \dots n$ runs

Table 4.5 — Performance Testing Results

ID	Operation	Runs	Average (ms)	Minimum (ms)	Maximum (ms)	Result
T055	POST /login (valid admin credentials)	20	57.04	35.14	117.43	PASS (<1s)
T056	GET /vendors (vendor list load)	20	29.06	17.04	38.54	PASS (<1s)
T057	GET /vendors/:id (single menu load)	20	31.26	23.09	41.10	PASS (<1s)
T058	GET /api/orders/mine (order-tracking poll)	20	13.67	8.72	19.01	PASS (<1s)
T059	GET /api/vendor/orders (vendor-dashboard poll)	20	16.04	9.17	24.99	PASS (<1s)
T060	GET /admin/dashboard (admin stats load)	20	26.08	19.48	33.95	PASS (<1s)
T061	POST /orders (order submission)	40	37.13	21.74	50.11	PASS (<1s)
T062	POST /vendor/orders/:id/advance (status update)	60	39.81	17.69	82.40	PASS (<1s)

What could not be reliably measured here: concurrent/load performance (e.g. 50 simultaneous students ordering at once) was not tested — the JSON-file store's `tx()` function performs a synchronous whole-file read-modify-write with no locking, so behaviour under concurrent write load is a real architectural question this test run does not answer. If a load test is required for the thesis, it should be run separately with a tool such as `autocannon` or `k6` and reported honestly rather than estimated.

4.7.6 Security Testing

Table 4.6 — Security Testing Results (13 checks; 12 passed, 1 informational finding)

ID	Check	Input	Expected	Actual	Status
T024	Rejected-vendor account access	Login after admin rejection	Ideally blocked	Finding: app only special-cases status="pending" at login, not "rejected" — a rejected vendor's session is still created, though their vendor profile row is gone so /vendor/dashboard then 404s	<i>FINDING</i>
T033	Unauthenticated order submission	POST /orders, no session cookie	Redirect to /login	302 → /login, no order created	PASS
T035	Order-list isolation between students	Student fetches /api/orders/mine	Only own orders returned	Confirmed — all returned orders belonged to the requesting student	PASS
T039	Cross-vendor order tampering	Vendor A attempts to advance Vendor B's order	No-op (vendorId ownership guard)	Status unchanged before/after — blocked as expected	PASS
T043	Student accesses vendor dashboard	GET /vendor/dashboard as student	403 Forbidden	403 as expected	PASS
T044	Student accesses admin dashboard	GET /admin/dashboard as student	403 Forbidden	403 as expected	PASS
T045	Vendor accesses admin dashboard	GET /admin/dashboard as vendor	403 Forbidden	403 as expected	PASS
T046	Unauthenticated access to student route	GET /vendors, no cookie	Redirect to /login	302 → /login	PASS
T047	Unauthenticated access to vendor API	GET /api/vendor/orders, no cookie	Redirect to /login	302 → /login	PASS
T048	Non-admin attempts vendor approval	Student POSTs /admin/vendors/:id/approve	403 Forbidden	403 as expected	PASS
T049	Password storage format	Inspect stored passwordHash	bcrypt hash, never plaintext	Confirmed bcrypt-format (\$2a\$08\$. . .), not equal to plaintext	PASS
T050	Stored XSS via vendor business name	businessName=<script>alert('xss')</script>	EJS auto-escapes output	Raw payload absent from rendered HTML; escaped entity form present	PASS
T051	Direct access to database file	GET /db.json and /data/db.json	404 — not web-accessible	Both 404 as expected	PASS

Additional source-level security observations (found by code inspection, not HTTP testing, and reported here rather than omitted): (1) `server.js` hardcodes the session secret as the literal string `"campusbites-demo-secret"` — acceptable for a demo, but must be replaced with a secret loaded from an environment variable before any real deployment. (2) The session cookie is not marked `secure` (reasonable in this HTTP-only local dev setup, but must be enabled once the app is served over HTTPS). (3) There is no rate-limiting on `/login` or `/register`, so brute-force / credential-stuffing attempts are not throttled. (4) SQL injection is not applicable — the data layer is a JSON file, not a SQL database — but the equivalent risk class (unsanitised user input reaching the persistence layer) was checked via the XSS test above and via manual review of `data/store.js`, which does simple property assignment with no query construction of any kind.

4.7.7 Usability / User Acceptance Test Data

This entire subsection is SYNTHETIC TEST DATA, clearly labelled as such in `testing/test-data/usability_test_data.csv` (column `data_label` = `SYNTHETIC_TEST_DATA` on every row). It was generated to demonstrate the analysis method and table structure Chapter 4 needs — it is **not** real participant feedback and must not be presented as such in your final submission. See §4.8.5 for exactly what real data you need to collect instead.

Table 4.7 — Usability Evaluation (synthetic demonstration data, 5-point Likert scale, 10 student + 4 vendor synthetic respondents)

Item	Respondents (n)	Average score / 5
Ease of registration/login	14	4.29
Ease of browsing menus (students only)	10	4.40
Ease of placing orders (students only)	10	4.30
Clarity of order status (students only)	10	4.30
Ease of vendor order management (vendors only)	4	4.50
Overall usefulness	14	4.21
Overall satisfaction	14	4.14

Item Average = (Σ Likert scores for item) / (number of respondents who answered that item)

4.8 Results and Discussion

4.8.1 Overall test statistics

Table 4.8 — Overall Testing Summary

Category	Total	Scored	Passed	Failed	Not applicable	Pass rate
Unit	21	21	20	1	0	95.2%
Integration	3	3	3	0	0	100.0%
System	5	3	2	1	2	66.7%
Negative/Error	12	12	12	0	0	100.0%
Security	13	12	12	0	1	100.0%
Overall	54	51	49	2	3	96.1%

Overall Pass Rate (%) = $(49 / 51) \times 100 = 96.1\%$

This figure should be read carefully in a viva: it counts only scored (pass/fail-capable) tests. The 3 "not applicable" items and the 2 genuine failures both point at the same two root causes — email format is not validated server-side, and vendor menu management (FR7) is not implemented — and are discussed below rather than folded silently into the headline number.

4.8.2 Discussion of findings

1. **Core order flow is solid.** Registration, login, role-based access control, vendor approval/rejection, menu browsing, order placement, order-total calculation, poll-based notification, and the full status lifecycle all behaved exactly as specified and were exercised with real, varied data (11 students, 10 active vendor businesses, 40 orders spanning all four statuses).
2. **Two real gaps exist between the Chapter Three design and the delivered system:** (a) server-side email format validation is missing (client-side only), and (b) FR7 (vendor menu management) was never built — this was a scope cut visible in `SPEC.md`'s own text ("Vendor menus are seeded from real data (not editable via UI in this build)"), so it is a known, documented limitation of the 5-hour demo build rather than a hidden defect.
3. **Database technology does not match Chapter Three.** The design specifies SQLite; the implementation uses a flat JSON file. This should be corrected in the write-up, or the implementation should be migrated, before final submission — examiners are likely to check this directly.

4. **Security fundamentals are in place** for a system of this scope: bcrypt password hashing, session-based auth, consistent role-based route guarding, output-encoding that defeats a basic stored-XSS probe, and correct ownership checks preventing students/vendors from reading or mutating each other's data. The one behavioural gap found (rejected vendors are not explicitly blocked from logging in, only pending ones) is a minor, fixable finding, not an exploitable data-access flaw, since the vendor's profile is deleted on rejection.
5. **Performance is not a concern at this scale.** All measured operations completed in well under 1 second on a single unloaded local instance (max observed: 117ms for a login round-trip). This says nothing about behaviour under concurrent load, which was explicitly not tested — see the note in §4.7.5.

4.8.3 Functional Requirement → Test Evidence Mapping

Table 4.9 — FR-to-Test Mapping

Requirement	Test IDs	Verdict
FR1 — User Registration & Login	T001–T020, T024	Implemented and verified for all three roles, including the vendor pending-approval gate. One informational gap: rejected (not pending) vendors are not explicitly blocked from logging in (T024).
FR2 — Menu Browsing	T025, T026, T027	Implemented and verified — active-vendor filtering and 404 handling both correct.
FR3 — Order Placement	T028–T033	Implemented and verified, including quantity handling, multi-item carts, and all tested negative paths.
FR4 — Order Notification	T036	Implemented as 4-second client-side polling, not a push notification. Verified an order becomes visible to the correct vendor without vendor-initiated refresh.
FR5 — Order Status Update	T037, T038, T039, T040	Implemented and verified, including a strict forward-only lifecycle and a cross-vendor tampering guard.
FR6 — Order Tracking	T034, T035	Implemented and verified, including per-student data isolation.
FR7 — Vendor Menu Management	T042, T052	Not implemented. No route or UI exists for a vendor to add, edit, or remove a menu item.
FR8 — Administrative Oversight	T021, T022, T023, T024, T041, T048, T053	Vendor approval/rejection and platform counters are implemented and verified. "Generate basic sales reports" is only partially met — the dashboard shows live counters and per-vendor order counts, with no revenue figures, date filtering, or exportable report.

4.8.4 Non-Functional Requirement → Test Evidence Mapping

Table 4.10 — NFR-to-Test Mapping

Requirement	Evidence	Verdict
-------------	----------	---------

NFR1 — Usability	§4.7.7 (synthetic questionnaire); client-side UX affordances observed in register.ejs (password-strength meter, inline email check)	Automated tests cannot measure real usability. Synthetic data only demonstrates the method — real data collection required (§4.8.5).
NFR2 — Performance	T055–T062	Met for single-user, unloaded, localhost conditions (all <120ms). Concurrent-load behaviour not tested — flagged as a risk given the non-atomic file store (§4.7.5).
NFR3 — Reliability	T040 (invalid order ID does not crash the server); general absence of unhandled exceptions across 150+ requests issued during this run	Basic reliability demonstrated for the paths tested. The file-based store's lack of locking under concurrent writes is an unverified architectural risk, not a confirmed defect.
NFR4 — Security	T024, T033, T035, T039, T043–T051; source-level observations on session secret and rate limiting (§4.7.6 note)	Strong on access control and output encoding. Two concrete improvements identified: server-side email validation, and moving the session secret out of source code.
NFR5 — Availability	None collected	Not testable from a local single-instance demo run. Availability/uptime requires a deployed environment with monitoring over time — see §4.8.6.

4.8.5 What you still need to collect manually

- **Real usability/UAT data:** Have at least 8–10 real students and 3–4 real vendor users actually use the running app (register, browse, order / manage orders) and fill in a Likert questionnaire using the same items as Table 4.7. Record their responses as `real_usability_data.csv` alongside (not overwriting) the synthetic file, and clearly mark it as real participant data with consent noted.
- **Screenshots for Chapter 4 / Appendix** (all easy to capture from your own browser against the already-running app): landing page; registration form for both roles; the "pending approval" login message for an unapproved vendor; the admin dashboard showing a pending approval and the approve/reject buttons; the vendor grid and a vendor's menu page; the cart/order submission; the student order-tracking page showing an order mid-status with the auto-refreshing badge; the vendor dashboard showing an incoming order and the "Accept/Mark ready/Mark picked up" buttons in sequence; and one browser dev-tools Network-tab screenshot showing a 4-second polling interval on `/api/orders/mine` or `/api/vendor/orders` as direct visual evidence for FR4.
- **Availability data (NFR5):** if your department expects an uptime figure, this requires deploying the app somewhere persistent (even a free-tier host) and monitoring it over days/weeks — it cannot be produced from a local test run.
- **A supervisor/committee decision on the two design mismatches** flagged in §4.4 and the FR7 gap in §4.8.2 — either correct the Chapter Three narrative to match what was actually

built, or extend the implementation (SQLite migration, vendor menu CRUD) before defence, depending on how much time remains.

4.8.6 Recommended Chapter 4 table numbering (as used above)

Table	Content
Table 4.1	Development environment
Table 4.2	Languages/frameworks/tools
Table 4.3	Chapter 3 vs implemented database entity comparison
Table 4.4	Pages/routes and their views
Table 4.5	Unit testing results
Table 4.6	Integration testing results
Table 4.7	System testing results
Table 4.8	Negative/error testing results
Table 4.9	Performance testing results
Table 4.10	Security testing results
Table 4.11	Usability evaluation (synthetic, clearly marked)
Table 4.12	Overall testing summary
Table 4.13	FR-to-test mapping
Table 4.14	NFR-to-test mapping

Note: the running numbering used inline above (4.1–4.10) groups a few of these together for readability; renumber to 4.1–4.14 if your department wants one table per numbered reference — the content above already maps 1:1 onto this list.

Generated from live test execution against CampusBites on 8 August 2026. Source data and reproduction steps: `testing/run-tests.js`, `testing/results/*.csv`, `testing/test-data/*.csv`, `testing/test_summary.md`.